



Research papers

Physics-informed neural networks for solving flow problems modeled by the 2D Shallow Water Equations without labeled data

Xin Qi^{*}, Gustavo A.M. de Almeida, Sergio Maldonado

Faculty of Engineering and Physical Sciences, University of Southampton, Southampton, SO16 7QF, Hampshire, United Kingdom

ARTICLE INFO

This manuscript was handled by Andras Barossy, Editor-in-Chief, with the assistance of John Chang, Associate Editor.

Keywords:

Physics-informed neural network
Fully connected neural network
Convolutional neural network
Shallow water equations
Free-surface flow

ABSTRACT

This paper investigates the application of physics-informed neural networks (PINNs) to solve free-surface flow problems governed by the 2D shallow water equations (SWEs). Two types of PINNs are developed and analyzed: a physics-informed fully connected neural network (PIFCN) and a physics-informed convolutional neural network (PICN). The PINNs eliminate the need for labeled data for training by employing the SWEs, initial and boundary conditions as components of the loss function to be minimized. Results from a set of idealized and real-world tests showed that the prediction accuracy and computation time (i.e., training time) of both PINNs may be less affected by the resolution of the domain discretization when compared against solutions by a Finite Volume (FV) model. Overall, the PICN shows a better trade-off between computational speed and accuracy than the PIFCN. Also, our results for the idealized problems indicated that PINNs can provide more than 5 times higher prediction accuracy than the FV model, while the FV simulation with coarse resolution (e.g., 10 m) can provide sub-centimeter accurate (RMSE) solutions at least one order of magnitude faster than the PINNs. Results from a river flood simulation showed that PINNs delivered better speed-accuracy trade-off than the FV model in terms of predicting the water depth, while FV models outperformed the PINNs for predictions of total flow discharge.

1. Introduction

Free-surface flow phenomena are usually modeled by the shallow water equations (SWEs), a nonlinear system of partial differential equations (PDEs) governing the evolution of water depth and vertically averaged velocity in the two horizontal dimensions. Over the last decades, significant efforts have been made to approximate the solution to the SWEs in a discretized form through numerical methods, such as Finite Difference (FD) (e.g., Casulli, 1990; Molls and Chaudhry, 1995; Kurganov and Levy, 2002), Finite Volume (FV) (e.g., Alcrudo and Garcia-Navarro, 1993; Bale et al., 2003; Botta et al., 2004; Yoon and Kang, 2004; Toro and Garcia-Navarro, 2007), or Finite Element (FE) (e.g., Lynch and Gray, 1979; Hanert et al., 2005; Dawson et al., 2006; Marras et al., 2016). These methods are now well-established and have been the object of extensive tests and validation (e.g., Toro and Garcia-Navarro, 2007; Wilson et al., 2007; Liang and Marche, 2009; LeVeque et al., 2011). While the ability of these models to capture the main properties of free-surface flows has been widely verified, accurate solutions to real-world problems typically require the use of a finely resolved computational mesh, which tends to significantly increase the computational cost (Bernard et al., 2009; Liang, 2011; Juez et al., 2014). In explicit numerical schemes for the solution of the 2D SWEs,

the computational cost C scales cubically with the size of the computational grid Δx (i.e., $C \sim \Delta x^{-3}$). As a result, important applications such as large-scale flood simulations are often beyond the capabilities of available numerical methods given existing computational resources. This is particularly challenging when simulations need to be performed in real-time, or as part of a probabilistic flood risk analysis (Leskens et al., 2014; Sanders and Schubert, 2019; Ferrari and Vacondio, 2022; Li et al., 2022). Although the computational speed of models can be improved by using state-of-the-art hardware and parallel computing algorithms (Leandro et al., 2014; Monnier et al., 2016), such techniques may still be insufficient to meet the computational requirements of many important applications (Kabir et al., 2020). Owing to these limitations, a cost-effective model for free-surface problems may offer an appealing alternative. To this end, an avenue worth exploring is Artificial Intelligence, as discussed next.

Over the last decades, Machine Learning (ML) models, and Artificial Neural Networks (ANNs) in particular, have found a wide range of applications in water-related problems, such as in flood simulation (Debbarma et al., 2024; Gurbuz et al., 2024), climate forecasting (Yeganeh-Bakhtiary et al., 2022; Donnelly et al., 2024a), wave prediction (Habib et al., 2023; Yeganeh-Bakhtiary et al., 2023), to cite

^{*} Corresponding author.

E-mail address: xq1n19@soton.ac.uk (X. Qi).

only a few. The extraordinary increase in applications of ML models is largely due to their ability to mathematically describe any nonlinear relationship between inputs and outputs according to the universal approximation theorem (Hornik et al., 1989), the increasing availability of data for training, and increasing computational power.

The first works using ML for the solution of problems governed by the SWEs are relatively recent and focused on the development of simple meta-models (e.g., Kabir et al., 2020; Liu and Pender, 2015; Bermúdez et al., 2019; Mahesh et al., 2022). In this type of model, ML is used to build a prediction model to describe the input–output relationship previously obtained through the solution of the governing PDEs by another numerical approximation model; i.e. the ML model thus becomes a surrogate model. These surrogate models typically need to be trained using the results of a large number of numerical simulations conducted at fine resolution, which can be very computationally demanding.

Physics-informed neural networks (PINNs), for which large datasets (and therefore computationally expensive numerical simulations) are not required for training, have gained increasing attention in recent years (Pang et al., 2019; Mao et al., 2020; Cai et al., 2021; Krishnapriyan et al., 2021; Jin et al., 2021; Kharazmi et al., 2021; Jagtap et al., 2023). A PINN is essentially a ML algorithm which uses the information contained in the physical laws (such as the governing PDEs, boundary and initial conditions) to train the model. This can reduce or even eliminate the need for training data and thus, expensive numerical simulations. In particular, training a data-free PINN (i.e. when labeled input–output data is not used to train the network) is required to only satisfy the governing PDEs, Initial Conditions (ICs) and Boundary Conditions (BCs) simultaneously. Recent applications of ML algorithms to solve complex physics phenomena have focused on the use of Deep Learning (DL) models (e.g., Sun et al., 2020; Zhang et al., 2020; Haghighat et al., 2020; Vlassis and Sun, 2021; Wu et al., 2023). DL is a form of ANN with more than one hidden layer, which provides the complexity required to model intricate nonlinear relationships (Voulodimos et al., 2018; Khan and Yairi, 2018; Rastgoo et al., 2021; Zhu et al., 2017). In the past few years, a large number of data-free PINNs have been developed by employing DL techniques, such as the Fully Connected Neural Networks (FCNNs) and Convolutional Neural Networks (CNNs). For example, in Raissi et al. (2019) several FCNNs were trained to predict the solutions to various systems of PDEs, including Allen–Cahn, Schrödinger, Navier–Stokes, and Korteweg–de Vries equations. In the context of fluid dynamics, other implementations of FCNNs include those of Sun et al. (2020) and Mao et al. (2020), who used their DL models to find solutions to the Navier–Stokes (in steady state) and Euler equations (involving shock waves), respectively. The use of CNNs has also been explored, for instance, for problems governed by the Navier–Stokes (Cai et al., 2021) and Boltzmann transport equations (Li et al., 2021), or for predicting steady flow in random heterogeneous media (Zhu et al., 2019). The success of these works shows that data-free PINNs should be considered as serious contenders for solving flow problems that are typically modeled by PDEs, and which have been traditionally solved using conventional numerical methods (FD, FV, etc.).

While PINNs trained from labeled data (i.e., mainly conventional numerical solutions) have been used to solve the SWEs (e.g., Mahesh et al., 2022; Feng et al., 2023; Li et al., 2023; Donnelly et al., 2024b; Fraehr et al., 2024), their model performance still relies on the quality and quantity of the given labeled data. A data-free PINN, trained without any labeled data, can be regarded as a completely independent method for solving the SWEs. However, to the authors' knowledge only a very limited number of articles (e.g. Bihlo and Popovych, 2022) has been published so far on the use of a data-free PINNs for this purpose. In Bihlo and Popovych (2022), a data-free PINN (specifically, based on a FCNN) was employed to find solutions to the SWEs on a spherical domain, and the focus was on idealized problems which may find applications in meteorology. Whether a similar DL technique

may be used to accurately and efficiently solve challenging free-surface flow problems involving friction, wet-dry front and complex boundary conditions, such as large-scale simulations of flood, remains an open question.

The solution of PDEs, and in particular of the SWEs, using PINN algorithms is still in its infancy and further investigation is required to understand the main characteristics of solutions obtained by these methods. Firstly, the trainset for PINNs needs to be generated from a particular, discrete, set of points. It remains unclear how accuracy and computational performance (i.e., training speed) depend on the discretization of the domain. Additionally, both FCNNs and CNNs are commonly used DL models in the research field of PINN. However, in a specific problem governed by a system of PDEs, it is usually difficult to determine which one will deliver the best performance before carrying out tests.

The aim of this paper is to develop and test two different PINN models to approximate solutions to various free-surface flow problems governed by the 2D SWEs. The PINN models are based on the FCNN and CNN approaches, and are hereafter referred to as PIFCN (physics-informed fully connected network) and PICN (physics-informed convolutional network), respectively. These models are data-free in that they do not require data from separate numerical simulations, or laboratory/field measurements, to train the networks. In this paper both PIFCNs and PICNs are compared against the Finite Volume (FV) solver of the 2D SWE developed by de Almeida et al. (2016) through a set of test cases including two idealized flow problems and one real-world flood event. The rest of this paper is organized as follows. First, the governing equations and the framework of both PINNs are described in Section 2. This section also provides a concise review of FCNNs and CNNs. In Section 3, the accuracy and computational performance of the proposed physics-informed networks (PIFCN and PICN) are investigated for the three test cases. The main outcomes of the study are discussed and summarized in Section 4.

2. Methods

2.1. Overview

Most problems requiring the simulation of free-surface flows in the horizontal plane, such as flow in rivers and estuaries, dam-breaks, and flood wave propagation can be modeled by the SWEs. The 2D SWEs represent a system of nonlinear, hyperbolic PDEs describing the conservation of water mass and depth-average momentum, which can be expressed as:

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}(\mathbf{U})}{\partial x} + \frac{\partial \mathbf{G}(\mathbf{U})}{\partial y} = \mathbf{S}(\mathbf{U}); \quad (1)$$

where

$$\mathbf{U} = \begin{bmatrix} h \\ hu \\ hv \end{bmatrix}, \quad \mathbf{F}(\mathbf{U}) = \begin{bmatrix} hu \\ hu^2 + \frac{1}{2}gh^2 \\ huv \end{bmatrix}, \quad \mathbf{G}(\mathbf{U}) = \begin{bmatrix} hv \\ huv \\ hv^2 + \frac{1}{2}gh^2 \end{bmatrix}, \quad \mathbf{S}(\mathbf{U}) = \begin{bmatrix} 0 \\ gh(s_{ox} - s_{fx}) \\ gh(s_{oy} - s_{fy}) \end{bmatrix}, \quad (2)$$

where x and y are the spatial (horizontal) coordinates; t is time; $h(x, y, t)$ is the water depth; $u(x, y, t)$ and $v(x, y, t)$ denote the x and y components of the depth-averaged flow velocity, respectively; $s_{ox} = -\partial z/\partial x$ and $s_{oy} = -\partial z/\partial y$ are the bed slopes in the x and y directions, respectively, and $z(x, y)$ is the terrain elevation (assumed constant in time); s_{fx} and s_{fy} denote the friction slopes in the x and y directions, respectively. The friction slopes can be modeled using Manning–Strickler's expression, $s_{fx} = n^2 u \sqrt{u^2 + v^2} h^{-4/3}$, $s_{fy} = n^2 v \sqrt{u^2 + v^2} h^{-4/3}$, where n is the Manning coefficient. Solutions $\mathbf{U} = (h, hu, hv)^T$ to this system (subject to well-posed boundary and initial conditions) can be computed by several numerical methods available, as discussed in the Introduction.

In this paper we propose a ML-based solution to this problem, whereby the input layer $\mathbf{x}$ represents the independent variables and parameters of the problem, $\mathbf{x} = (x, y, t, n, z)$, and the trained model $\mathfrak{N}$ is expected to provide an approximate solution for $h(\mathbf{x})$, $hu(\mathbf{x})$ and $hv(\mathbf{x})$ in the corresponding domain; in other words:

$$\mathbf{U}(\mathbf{x}) \cong \tilde{\mathbf{U}}(\mathbf{x}) = \mathfrak{N}(\mathbf{x}; \Gamma), \quad (3)$$

where $\tilde{\mathbf{U}}(\mathbf{x})$ denotes the output from the PINN, which is in turn defined by the group of trainable parameters Γ (e.g., convolutional filter, weights and biases). The PINN models proposed in this paper are trained by minimizing the composite loss function, defined as:

$$\mathcal{L} = \lambda_1 \cdot \mathcal{L}_p + \lambda_2 \cdot \mathcal{L}_b + \lambda_3 \cdot \mathcal{L}_0, \quad (4)$$

where $\mathcal{L}$ (a scalar) is the loss function to be minimized, λ_{1-3} are the vectors of penalty coefficients for every specific loss term; namely, $\mathcal{L}_p$ penalizes the residuals of the SWEs, $\mathcal{L}_b$ and $\mathcal{L}_0$ penalize the BCs (subscript b) and ICs (subscript 0) residuals, respectively. These loss terms are in turn given by:

$$\mathcal{L}_p = \frac{1}{N} \sum_{i=1}^N |\partial_t \tilde{\mathbf{U}}_i + \partial_x \mathbf{F}(\tilde{\mathbf{U}}_i) + \partial_y \mathbf{G}(\tilde{\mathbf{U}}_i) - \mathbf{S}(\tilde{\mathbf{U}}_i)| \quad (5a)$$

$$\mathcal{L}_b = \frac{1}{N_b} \sum_{i=1}^{N_b} |\tilde{\mathbf{U}}_{b,i} - \mathbf{U}_{b,i}| \quad (5b)$$

$$\mathcal{L}_0 = \frac{1}{N_0} \sum_{i=1}^{N_0} |\tilde{\mathbf{U}}_{0,i} - \mathbf{U}_{0,i}| \quad (5c)$$

The tilde symbol ($\sim$) denotes neuronal network output and the subscript $i \in [1, N]$ refers to the i th collocation point. N represents the number of collocation points, which in this paper is defined from a uniformly discretized domain of the independent variables such that $N = n_x \times n_y \times n_t$, where n_x , n_y and n_t are the number of points used to discretize the domain along the x , y and t coordinates, respectively. The boundaries of the spatio-temporal domain are represented by a subset of N ; in particular, the model will employ $N_b < N$ and $N_0 < N$ points to define the BCs and ICs, respectively. Therefore, the boundary and initial conditions of the problem are set via selected collocation points, with the loss function at these points evaluated using Eqs. (5b) and (5c), respectively.

For each approximate solution produced by the PINN, the partial derivatives in Eq. (5a) are computed through the method of automatic differentiation (autodiff) (Paszke et al., 2017), which back-propagates derivatives from the outputs to the targeted inputs through the chain rule to compute the desired derivatives (Cai et al., 2021; Baydin et al., 2018). Thus, the partial derivatives of the approximate solution with respect to the independent variables can be computed without the errors common to numerical differentiation techniques. The loss function is minimized using the gradient descent method, with gradients of the loss function with respect to trainable parameters computed by back-propagation. These parameters can be updated either using all, or a subset (batch) of the collocation points.

One significant difficulty of solutions to flow problems modeled by the SWEs is the so-called wet-dry front issue (i.e., moving boundary). Physically, the value of the flow depth h cannot be negative. Areas of the domain where such solutions may be obtained correspond to dry areas, which are not governed by the SWEs. To overcome this problem, we set $\mathcal{L}_p = 0$ if the predicted value of $\tilde{h}$ is negative. This ensures that the model does not penalize predictions outside the wet domain.

Fig. 1 shows a diagram illustrating the overall modeling framework proposed in this paper for solving the SWEs by a PINN method. Note that the collocation points can be chosen randomly in the space-time domain and their number prescribed. The general steps are outlined below.

1. Generate the set of collocation points used to discretize the domain, including the subset of points used to define the BCs and ICs;

2. Define the architecture of the PINN;
3. Initialize the hyperparameters for the PINN;
4. Compute the outputs from the PINN with given inputs;
5. Compute the derivatives with respect to x, y, t and the corresponding loss $\mathcal{L}$
6. Update the PINN based on $\mathcal{L}$;
7. Repeat steps 3 to 5 until the end of the user-prescribed number of training epochs.

2.2. Fully connected neural network

The FCNN is the most commonly applied ML model and often includes more than one hidden layer. Every hidden layer receives the signals from the previous layer, performs basic computations defined at each neuron, and passes the results to the next layer (Haykin, 2009). Fig. 2 shows a diagram of a FCNN. Mathematically, the basic function of the output for the j th hidden layer $\mathbf{y}_j$ is:

$$\mathbf{y}_j = \varphi(\mathbf{W}_j \mathbf{y}_{j-1} + \mathbf{b}_j) \quad (6)$$

where $\mathbf{W}$ is the matrix of weights, $\mathbf{b}$ is the vector of biases and $\varphi()$ is the activation function.

In the proposed method, solutions for each output variable $\eta(\mathbf{x}) = h(\mathbf{x}) + z$, $hu(\mathbf{x})$, $hv(\mathbf{x})$ are approximated by 3 separate FCNNs with the same structure, as illustrated in Fig. 2. Every sub-FCNN receives the same raw inputs. As a result, the trainable parameters of the solution for each output variable are decoupled. This can significantly improve the prediction accuracy in multivariate problems, especially when the distributions and magnitudes of the variables are significantly different (e.g., Sun et al., 2020; Gao et al., 2021; Guo et al., 2020).

2.3. Convolutional neural network

The CNN adds one or more convolutional layers that extract features of the raw training dataset before feeding this onto the typical hidden layers used to build FCNNs. The general expression for the convolution operator $\star$ with 1 stride is:

$$(\mathbf{s} \star \mathbf{k})_i = \sum_{j=1}^n k_j s_{i+j-1} \quad i = 1, 2, \dots, m - n + 1 \quad (7)$$

where $\mathbf{s}$ denotes the input signal vector of length m (in this paper, this is $\mathbf{x}$), and $\mathbf{k}$ denotes the trainable filter of length n . The convolution operation is to slide the preset convolutional filter over the signal input and output the signal with a shorter length (i.e., the input vector is shortened by $n-1$ elements). The shorter length of the convolved output signal allows the following typical hidden layer to have fewer neurons, facilitating the network's learning of large-scale problems with high complexity (Gao et al., 2021).

Fig. 3 shows the structure of the CNN used in this paper. The trainset is generated from a number of points (i.e., collocation points) randomly sampled from a grid of equally spaced points. Each output variable, $h(\mathbf{x})$, $hu(\mathbf{x})$, $hv(\mathbf{x})$, is also predicted by a separate sub-CNN.

2.4. PINN design

The accuracy and computational performance of the PINNs described in the previous sections will be assessed and compared against the corresponding performance and solutions by a conventional FV model. There is currently no universal design approach to determine the optimal, or even appropriate, structure for a neural network (Bihlo and Popovych, 2022). The general selection rule for PINN design is to find a structure with the lowest possible complexity that achieves the desired accuracy of prediction. This rule can usually help provide an AI model with quick learning speed and improved prediction capabilities while avoiding overfitting issues (Blumer et al., 1987). In this paper, the final decision for the model structure (i.e. hyperparameters such

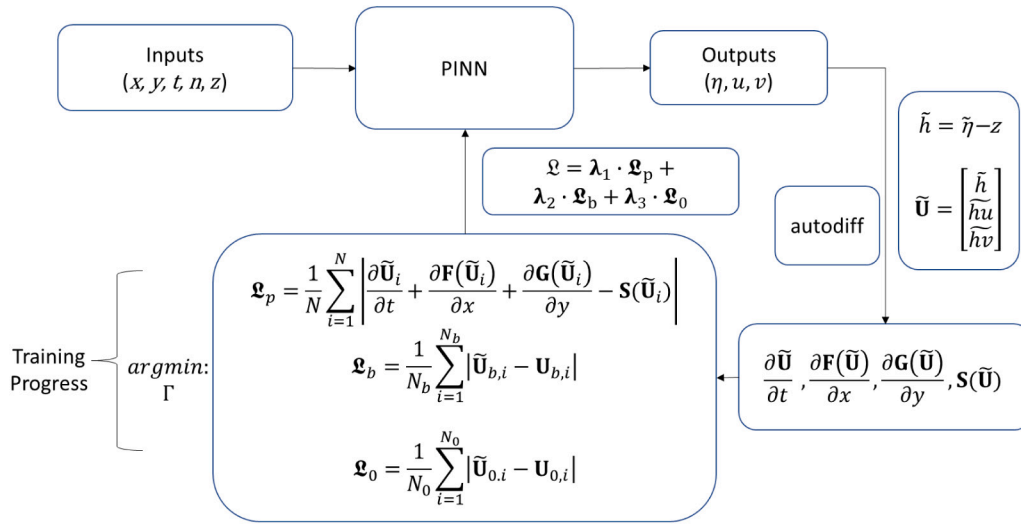


Fig. 1. A schematic diagram of a physics-informed neuronal network (PINN) for finding approximate solutions to the shallow water equations.

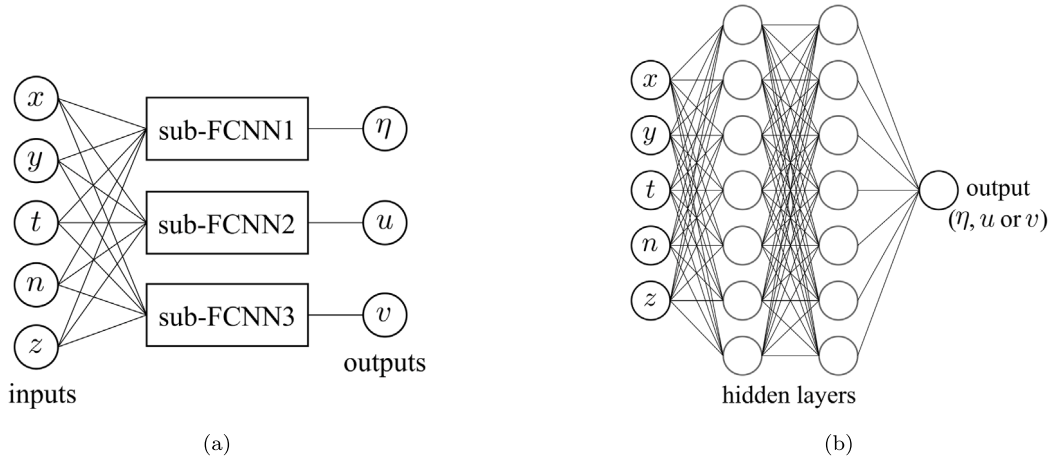


Fig. 2. (a) The architecture of the physics-informed fully connected networks (PIFCNs) employed in this paper. (b) An example of a typical fully connected neuronal network (FCNN) which is employed as a sub-network within the PIFCN to predict each individual output; as illustration, 2 hidden layers with 7 neurons each are shown, but these hyperparameters are varied in this study.

as the number of neurons, hidden layers, and convolutional layers and channels in the case of CNN) was made after many practical attempts (see Appendix A). As the evaluation of the PINNs performance in this paper consists of two, often competing, criteria (accuracy and computational cost), it may be difficult to find a single assessment metric to guide the PINNs design. Hence, we give priority to accuracy by gradually increasing the complexity of the PINNs until similar or higher accuracy than benchmark results (e.g. from an analytical solution or a finely resolved FV simulation) is attained. Generally, in our design iterations, the number of hidden layers and the corresponding neurons for building PINNs (i.e., PIFCN and PICN) started from 1 and 50, respectively. The number of convolutional layers and corresponding channels started from 1 and 5, respectively. For both PICN and PIFCN, we use the hyperbolic tangent activation function (Tanh). Note that the PINN design may change significantly depending on domain and flow conditions; i.e., it can be very problem-specific. Specific characteristics regarding the architecture of the networks used to solve each problem in this paper (and the corresponding computational time) are provided for each individual test in subsequent sections. It is also important to recognize that the networks chosen do not represent the strictly optimal structure, but only the best out of the subset of structures that were tested. Moreover, the predictive accuracy of PINNs may also be sensitive to the random initialization of its parameters.

For improving the learning speed and reducing the effect of parameter initialization, the Batch Normalization method of Ioffe and Szegedy (2015) was used, which normalizes the signals between adjacent convolutional or hidden layers. The Adam optimizer (Kingma and Ba, 2014), along with the '1-cycle' (Smith and Topin, 2019) strategy was used to control the training of the PINNs. The PINNs were implemented on the Pytorch platform (Paszke et al., 2017). The FV model used in this paper solves the 2D SWEs using a Godunov-type scheme applied to unstructured triangular meshes (further details of the methods are described in de Almeida et al. (2016)). The FV simulation and the training of the PIFCNs and PICNs were performed using the same cluster of computer nodes from the University of Southampton's supercomputer Iridis 5. Every computation was implemented on one computer node ensuring that the exact same hardware resources were employed (thus ensuring a fair comparison across all simulations performed).

3. Results and discussion

This section describes three case studies used to test the PINNs, comparing their results against analytical and numerical (Finite Volume) solutions. The first and second tests are idealized 1D (unsteady and steady, respectively) flow problems for which analytical solutions are available. However, simulations were performed on a 2D domain

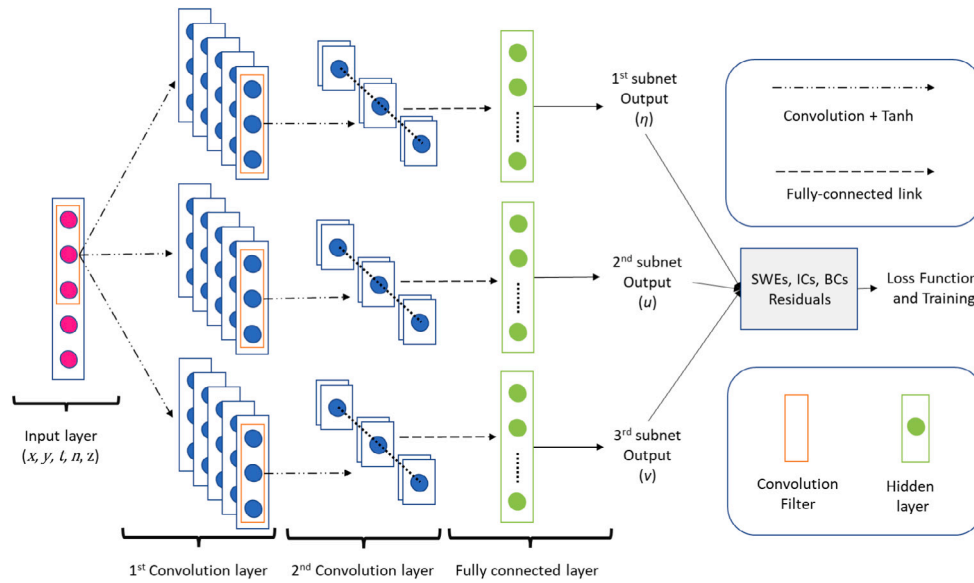


Fig. 3. An example of the structure of a CNN-based model with 3 subnets for solving free-surface flow problems. Each output variable (η , u or v) is approximated by a separate CNN with the above structure; all sub-networks receive the same inputs. Each CNN has two convolutional layers and one hidden layer. The hyperparameters shown in the figure are discussed in Section 2.4.

since the ultimate aim is to employ the PINNs developed here in 2D flow problems. The third test case is an unsteady two-dimensional simulation of a real-world flood event that took place in the Tiber river, Italy. This case study has been previously employed to evaluate the performance of other numerical models (e.g., Morales-Hernández et al., 2016; Shamkhalchian and de Almeida, 2021).

Topographic data used in all tests are defined by square grids with different resolutions. The grid points are used to generate a triangular mesh for the FV model. These are also employed, along with defined temporal steps, as the collocation points for the PINNs training. The accuracy of the solutions will be assessed by the root mean square error, $\mathcal{R}$, of the outputs of each model relative to the benchmark solution. For example, in the evaluation of accuracy for the prediction of h with N_p output points, the performance metric is defined as $\mathcal{R}_h = \sqrt{\sum (h_i - \hat{h}_i)^2 / N_p}$, where h_i is the benchmark solution (i.e., the analytical solution when available, or the solution of the FV model at fine resolution). The second performance metric we employ is the computational cost, $\mathcal{T}_c$, which represents training time for the PINNs (PICN and PIFCN), and run time for the FV model. In the results presented in the following sections, predictions by the FV, PIFCN and PICN models are labeled with the different spatial resolutions used. For example, FV (10) represents a 10 m resolved simulation using the FV hydraulic model, and PICN (50) refers to the prediction of the PICN trained from a 50 m resolved dataset. The temporal resolution of collocation points used to train the PINN models was defined through several tests in which the accuracy of the results was analyzed as a function of the time resolution. A point of diminishing returns (i.e. when further refinements of the temporal resolution do not translate into substantial accuracy improvements) was chosen for each of the tests described in this section. The PINN and FV simulations were not conducted using the same ranges of spatial resolutions since the main focus of this study is on the trade-offs between accuracy and computational performance, which differed substantially as a function of the resolution for the two types of models. In other words, while the resolution affects both accuracy and computational performance, it does so in a different way for the two methods compared. We have thus set ranges of resolutions that make final results comparable in terms of the aforementioned trade-off. Other considerations include hardware limitations for training PINNs at the finest resolutions used by the FV model, and the latter's inability to run at very coarse resolutions.

In the following sections, we present the benchmark solutions and the corresponding prediction errors in the same figure (e.g., Figs. 5, 12 and 9(a)). Note that all the actual predictions compared against the reference results are also presented in the 'Supplementary Material' document.

3.1. Flood wave propagation over a horizontal plane

3.1.1. Case description and model setup

The first test case is a one-dimensional simulation of an inundation wave propagating over a horizontal bed. A time-dependent BC is imposed at $x = 0$. Under the idealized assumption of a flow velocity that is constant in space and time, the problem admits an analytical solution to the 1D SWE, which can be expressed as Hunter et al. (2005):

$$h_a(x, t) = \left\{ \frac{7}{3} (n^2 u^2 (x - ut)) \right\}^{3/7}, \quad (8)$$

where the subscript a is used to denote the analytical solution. The domain used is a 100 m wide, 1200 m long channel. The constant velocity is set as $u(x, t) = 0.29 \text{ ms}^{-1}$ and the boundary condition $h(x = 0, t)$ is given by Eq. (8). The domain is initially dry, i.e., $h(x, t = 0) = 0$. Manning's coefficient n is set to $0.03 \text{ s m}^{-1/3}$. The duration of the simulation is 3600 s. The FV model was run at resolutions of 1, 2, 5 and 10 m, while the PINN models were trained with datasets defined at resolutions of 10, 25, 50 and 100 m. While the time step of the explicit FV scheme is controlled by the Courant–Friedrichs–Lewy (CFL) stability condition, the regression approximation implemented by the PINN model is not limited by temporal resolution. However, the time step adopted to train the PINN model is a factor that clearly affects both accuracy and computational performance. For this test, we use a temporal resolution for the PINN of 300 s. The batch size used was the full set of collocation points $n_x \times n_y \times n_t$.

The architecture of the PICN consists of 2 convolutional layers (the first and second layers have 5 and 20 channels, respectively) and 1 fully connected hidden layer with 50 neurons. The architecture for the PIFCN consists of 3 fully connected hidden layers, each of which has 1000 neurons.

3.1.2. Results and analysis

Figs. 4 give some examples of the predictions of h compared against the analytical solutions at $t = 1800 \text{ s}$. Figs. 5 and 6 illustrate the

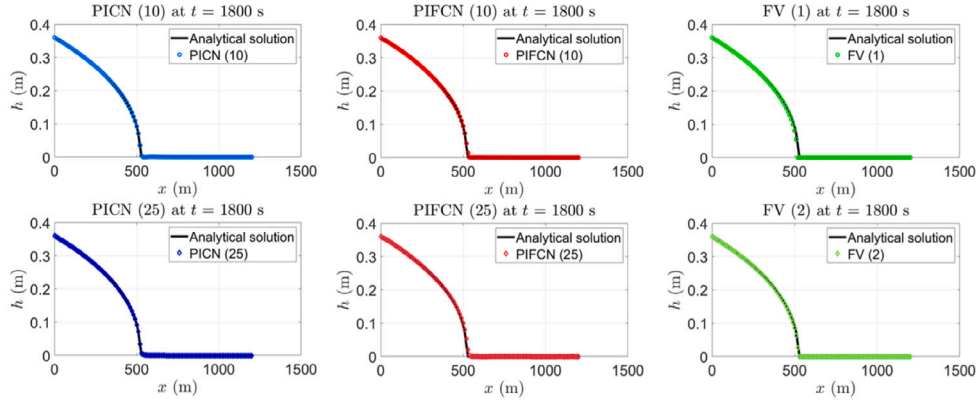


Fig. 4. Examples of longitudinal profiles ($y = 50$ m) of water depth h obtained by each PICN, PIFCN and FV against the analytical solution (black line) at $t = 1800$ s.

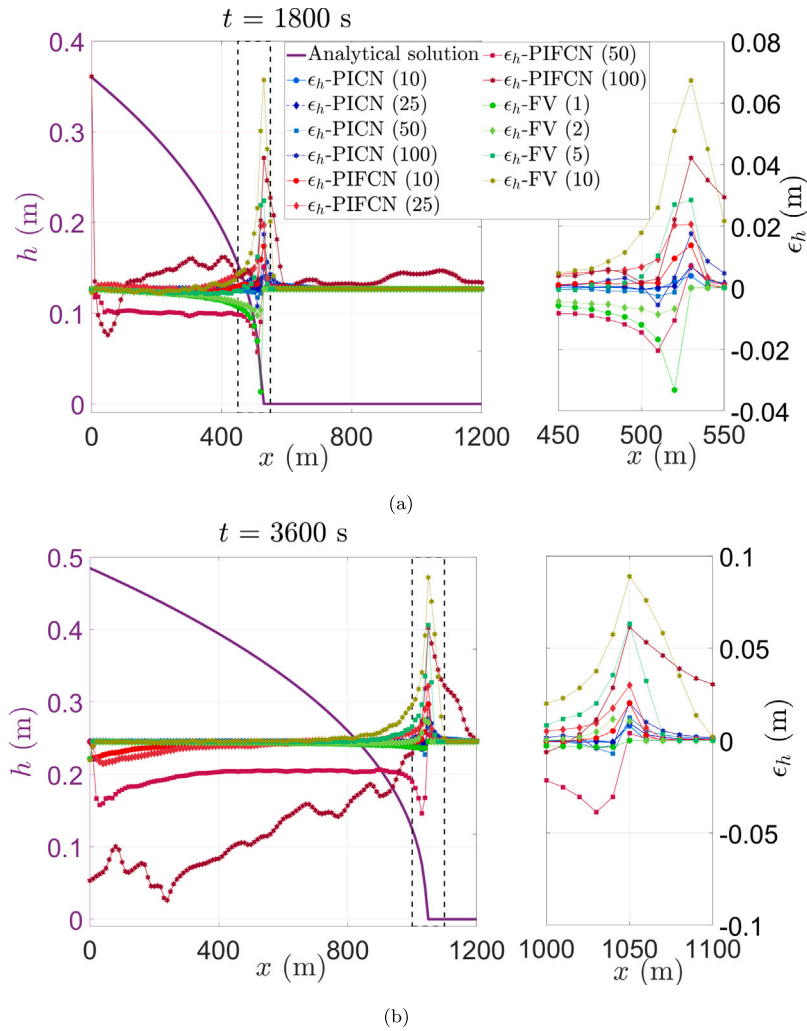


Fig. 5. Test 1: Longitudinal profiles ($y = 50$ m) of water depth errors ϵ_h relative to the analytical solution obtained by each of the models at $t = 1800$ s (a) and 3600 s (b), shown against right y -axis. The analytical solution for h (purple line) is plotted against the left y -axis; note that the right-side figure is the enlarged version of the rectangular box in the left-side figure; both figures share the same right y -axis. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

analytical solutions of $h(x, y = 50$ m) and $hu(x, y = 50$ m) (left vertical axes), and the corresponding error (right vertical axes) $\epsilon_h(x, y = 50$ m) = $\tilde{h}(x, y = 50$ m) - $h_a(x, y = 50$ m) and $\epsilon_{hu}(x, y = 50$ m) = $\tilde{hu}(x, y = 50$ m) - $(hu)_a(x, y = 50$ m) computed by all three models at $t = 1800$

and 3600 s, respectively. Values of hu are not reported as the test case is fundamentally one-dimensional. Overall, all the water depth predictions, with the exception of PIFCN (100), show good agreement with the analytical solution (i.e. most results displaying $|\epsilon| < 0.01$ m). As the

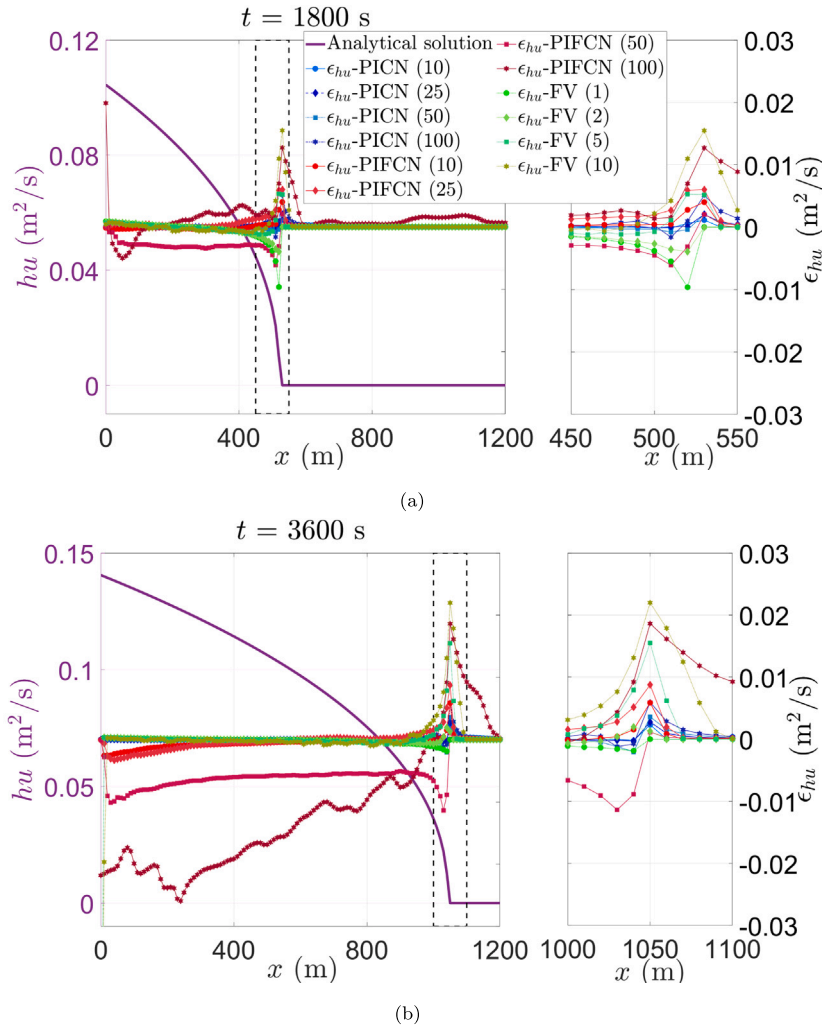


Fig. 6. Test 1: Longitudinal profiles ($y = 50$ m) of water discharge errors ϵ_{hu} relative to the analytical solution obtained by each of the models at $t = 1800$ s (a) and 3600 s (b), shown against right y-axis. The analytical solution for h_u (purple line) is plotted against the left y-axis; note that the right-side figure is the enlarged version of the rectangular box in the left-side figure; both figures share the same right y-axis. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

position of the wet-dry front predicted by the models does not exactly match the analytical solution, and the front is steep at that point, errors are larger in this region. The largest prediction errors (of both h and h_u) near the wet-dry front are from FV(10). The finely resolved PICN (e.g. 10 m and 25 m) produced the most accurate solutions across all models tested, as is further discussed in the following paragraph. Upstream from the front, PICN and FV both display similar prediction accuracy of both h and h_u , whereas PIFCNs with coarsely resolved trainsets (i.e., 50 m and 100 m) provide higher prediction errors of h_u .

Fig. 7 shows $\mathcal{R}_h$ (relative to the analytical solution h_a) for all results obtained with the PICN, PIFCN, and FV models as a function of the corresponding computational time $\mathcal{T}_c$. Table 1 also summarizes the results for each model. The sum to compute $\mathcal{R}_h$ is over all collocation points; i.e., spanning the whole spatio-temporal domain. In this figure, the various points (blue and red) presented for each PINN model represent solutions obtained at different epochs during the training of the networks, which correspond to different computation time and level of accuracy. The green cross points represent the simulation accuracy and computation time for the FV model. The results in this figure are based on model (i.e. PICN, PIFCN, and FV) outputs at the same grid points selected from the entire domain with a spatial and temporal resolution of 10 m and 360 s. Predictions of h_u follow the general pattern observed for h on Fig. 7 and are not reported here to avoid repetition. Fig. 7 allows us to comparatively assess the performance of

the models tested in terms of their speed-accuracy trade-off. Based on this criterion, a model performs better than another when it provides more accurate results under the same computational time, or vice-versa; in other words, the best results are those closest to the bottom left corner of the plot.

Fig. 7 shows that FV (10) and FV (5) produce sub-centimeter $\mathcal{R}_h$ (which is usually considered a good level of accuracy for many applications) at least one order of magnitude faster than the PINN models, whereas FV (2) takes slightly longer than PICNs (for the same level of accuracy), and FV (1) only outperforms PIFCN (10) in terms of the speed-accuracy trade-off. All PINNs except PIFCN (100) show the potential to achieve higher accuracy of prediction than the FV model at the highest resolution tested here (1 m), provided they are trained for long enough. PICNs provide a faster solution (for similar $\mathcal{R}_h$ values) than PIFCNs. Also, for PIFCN, the trainset size (which in this case is determined by the resolution) did not significantly affect its maximum accuracy at resolutions ≤ 50 m, whereas the accuracy of the FV model continues to improve as the mesh is refined below 10 m.

3.2. Subcritical steady flow over an undulating bed

3.2.1. Case description and model setup

The second test case represents a 1D, steady, non-uniform flow over an undulating bed, for which an analytical solution is available (see

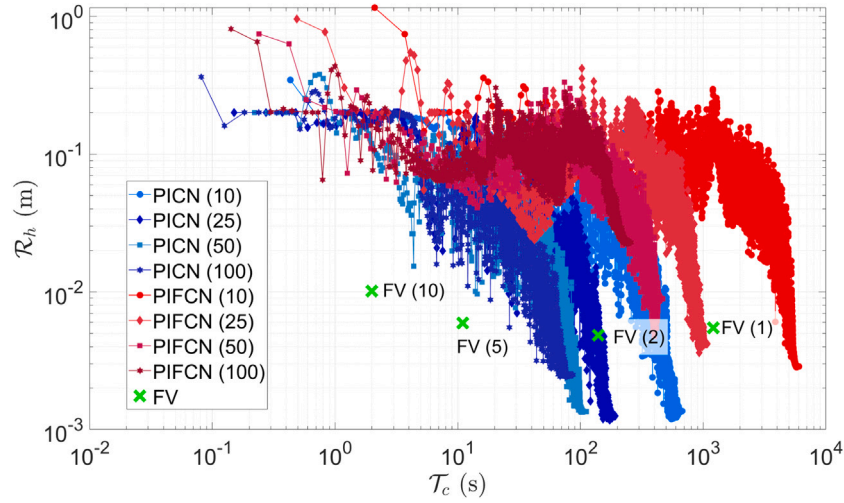


Fig. 7. Test 1: Values of R_h as a function of T_c (training time for PICN and PIFCN and run time for FV); note that the right-most point of each cloud corresponds to the highest accuracy that any given PINN can achieve. The number in brackets represents the resolution (in meters) of the training data set (for PICN and PIFCN) or mesh (for the FV model).

Table 1

Computation time and prediction accuracy of PICN, PIFCN and FV in the test 1. Note that the value of T_c indicates the training time when PINN achieve the optimal prediction accuracy.

Model	test1		
	T_c (s)	R_h (m)	R_{hu} (m ² s ⁻¹)
PICN(10)	674	1.4×10^{-3}	3.9×10^{-4}
PICN(25)	192	1.3×10^{-3}	3.6×10^{-4}
PICN(50)	111	1.4×10^{-3}	3.9×10^{-4}
PICN(100)	87	2.5×10^{-3}	7.2×10^{-4}
PIFCN(10)	6301	2.9×10^{-3}	8.5×10^{-4}
PIFCN(25)	1077	4.2×10^{-3}	1.0×10^{-3}
PIFCN(50)	467	4.7×10^{-3}	1.4×10^{-3}
PIFCN(100)	258	2.3×10^{-2}	5.7×10^{-3}
FV(1)	1206	5.4×10^{-3}	2.7×10^{-3}
FV(2)	140	4.8×10^{-3}	2.7×10^{-3}
FV(5)	11	5.9×10^{-3}	2.7×10^{-3}
FV(10)	2	1.0×10^{-2}	3.3×10^{-3}

Table 2

Computation time and prediction accuracy of PICN, PIFCN and FV in the test 2. Note that the value of T_c indicates the training time when PINN achieve the optimal prediction accuracy.

Model	test2		
	T_c (s)	R_h (m)	R_{hu} (m ² s ⁻¹)
PICN(10)	996	8.9×10^{-4}	1.9×10^{-6}
PICN(25)	381	1.4×10^{-3}	1.6×10^{-6}
PICN(50)	375	1.4×10^{-3}	4.2×10^{-6}
PIFCN(10)	2573	1.2×10^{-3}	5.6×10^{-7}
PIFCN(25)	1232	1.8×10^{-3}	9.0×10^{-7}
PIFCN(50)	759	1.1×10^{-3}	6.1×10^{-7}
FV(2)	1860	1.7×10^{-3}	4.8×10^{-3}
FV(5)	134	4.2×10^{-3}	1.2×10^{-2}
FV(10)	17.5	8.4×10^{-3}	2.4×10^{-2}

MacDonald, 1996; de Almeida and Bates, 2013; Delestre et al., 2013). This test case will be used to evaluate the solution obtained by the PICN and PIFCN in a problem with variable topography. The (rectangular) channel is 1000 m long, and Manning's coefficient n is set to $0.03 \text{ s m}^{-1/3}$. The constant inflow discharge per unit width of the channel is $q_x = uh = 2 \text{ m}^2 \text{ s}^{-1}$, and the downstream water depth is $\frac{9}{8} \text{ m}$. We prescribe the following function representing the water depth $h(x)$ (which is the benchmark solution against which the PINN approximations will be compared):

$$h(x) = \frac{9}{8} + \frac{1}{4} \sin\left(\frac{\pi x}{500}\right). \quad (9)$$

We model this 1D problem in a 2D domain using a width of 50 m (and $q_y = 0$) for the reasons discussed previously. Also, although the solution sought is for a steady flow problem, the steady condition was reached via an unsteady flow simulation, as the object of this paper is to test approximate methods to solve the time-dependent SWEs. The unsteady simulations were run from an initially dry domain over a period of 20 h, whereby the upstream BCs increase linearly with time from zero to the aforementioned constant values over the first 10 h of the simulation.

The training dataset for PICN and PIFCN was obtained from grids resolved at 5, 10, 25 and 50 m at the following times: 0, 1, 3, 5, 10, 15 and 20 h. The selected batch size is $2/7 \times N$, where the value $2/7$ comes from trial and error (larger batch sizes decreased the accuracy of the results). The FV model was run at resolutions of 2, 5 and 10 m.

For this case, the architecture of the PICN consists of 2 convolutional layers (the first and second layers have 5 and 20 channels, respectively) and 1 fully connected hidden layer with 50 neurons (same as in Test 1). The architecture of the PIFCN consists of 3 fully connected hidden layers, each of which has 1000 neurons (same as in Test 1).

3.2.2. Results and analysis

Fig. 8 gives some examples of the predictions of h compared against the analytical solutions at the center of the channel $h(x, y = 30 \text{ m})$. As the analytical solution is for the steady state, only the results at the end of the simulations are assessed. Fig. 9(a) shows the analytical curve for depth profile (left axis) and the corresponding errors of each of the approximate solutions ϵ_h (right axis) predicted by the PICN (blue points), PIFCN (red points), and FV models (green points). Fig. 9(b) presents similar results but for the variable hu . Overall, all models tested delivered results at sub-centimeter level of accuracy for h . The three PICNs showed the lowest errors of both h and hu , followed by FV (2). Values of ϵ_{hu} obtained from FV models display small (mostly within 1% of the actual value of hu) spatial variations, while they nearly are constant for both PIFCN and PICN.

Fig. 10 presents the values of R_h against the corresponding computational time taken to train the PICN (blue points), PIFCN (red points), and to run the FV model (green cross points) at different resolutions. Table 2 also summarizes the similar results for each model. The value of R_h of each model is calculated from its steady-state predictions of h ; namely: $R_h = \left(\sqrt{\sum (h_i - \bar{h}_i)^2 / N_p} \right) \Big|_{t=t_s}$, where t_s is the time after which a steady state is reached for each PINN or FV model. For the

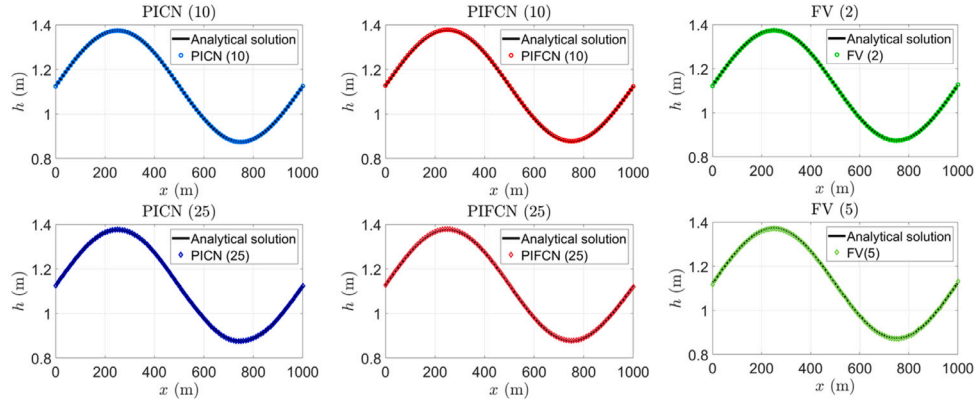


Fig. 8. Examples of longitudinal profiles ($y = 30$ m) of water depth h obtained by each PICN, PIFCN and FV against the analytical solution (black line) at the end of the simulation/training.

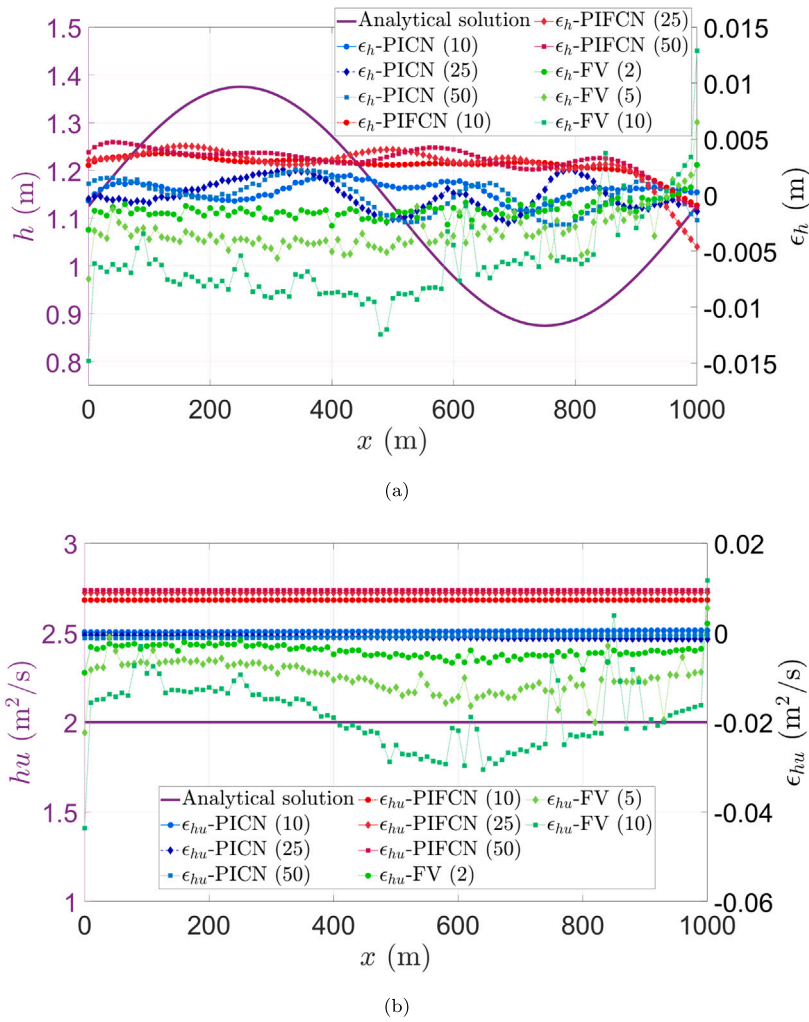


Fig. 9. Test 2: Longitudinal profiles ($y = 30$ m) of water depth errors ϵ_h (a) and water discharge errors ϵ_{hu} (b) obtained by each of the models at the end of the simulation/training (right y-axis). The analytical solution h (purple line) is plotted against the left y-axis. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

computation time of FV models described in Fig. 10, the value of $\mathcal{T}_c$ is the time required for all FV models to reach steady state. The results for hu show a pattern similar to that in Fig. 10 and are not presented for conciseness. All simulations achieve sub-centimetric $\mathcal{R}_h$, with FV (10) delivering the results at least one order of magnitude faster than the other solutions. PIFCN (10) was the slowest of all models. Fig. 10 shows

that the prediction of h from PICN (10) displays the highest accuracy, with an $\mathcal{R}_h$ of 0.85 mm, although this was obtained at a computation time that was 56 times longer than FV (10). All the PINN results also attain an accuracy higher than or similar to that of FV(2). In this test case, the relative differences in the prediction accuracy among the PINN models is less than the difference observed from FV (5) to FV (10). In

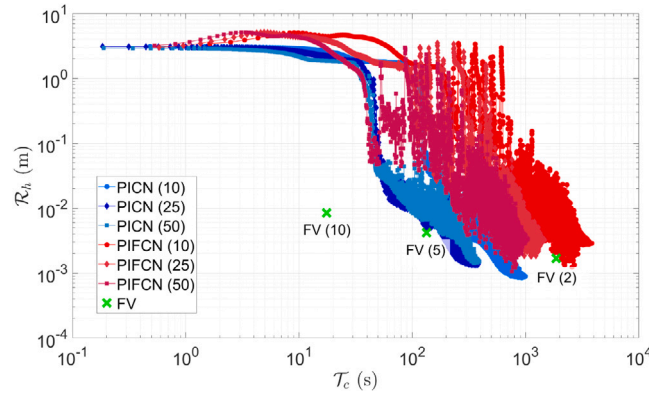


Fig. 10. Test 2: Values of R_h as a function of T_c (training time for PICN and PIFCN and run time for FV); note that the right-most point of each cloud corresponds to the highest accuracy that any given PINN can achieve. The number in brackets represents the resolution (in meters) of the training data set (for PICN and PIFCN) or mesh (for the FV model).

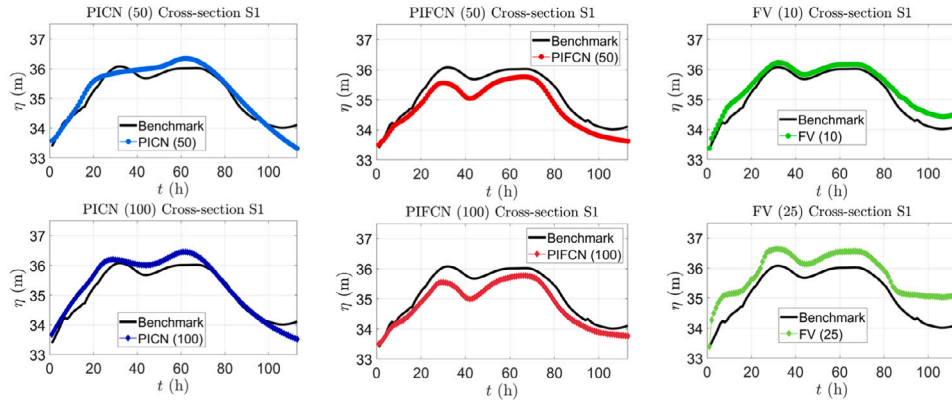


Fig. 11. Test 3: Examples of water surface elevation η obtained by each PICN, PIFCN and FV against the corresponding benchmark (black line) generated from FV (5) at cross-sections S1.

terms of the influence of resolution on the computational speed, the PICN is also less sensitive than PIFCN in this problem.

3.3. Simulation of real-world river flooding

3.3.1. Case description and model setup

While Tests 1 and 2 have assessed the ability of PINN models to deal with important aspects of flow problems, such as unsteadiness and variable topography, both case studies represented idealized, one-dimensional problems. In order to investigate the performance of PINNs under more complex and realistic problems, this section presents the results of simulations of a real-world scenario. The scenario in question is a flood event that occurred between 27 November and 1 December 2005 in the Tiber river (Morales-Hernández et al., 2016), which flows from the Apennine Mountains to the Tyrrhenian Sea in Italy. The reach of river employed in this simulation is approximately 6 km long and is located near the city of Rome. In this region, the mean discharge of the Tiber river is $267 \text{ m}^3 \text{ s}^{-1}$, while its peak discharge for a 200-year return period is around $3200 \text{ m}^3 \text{ s}^{-1}$. The event modeled in this paper was also previously simulated in Morales-Hernández et al. (2016) and Shamkhalchian and de Almeida (2021). The domain comprises an area of $6 \text{ km} \times 2 \text{ km}$. The duration of the event simulated is 113 h. The values of Manning's coefficient n used are the same as in Morales-Hernández et al. (2016) and Shamkhalchian and de Almeida (2021); namely, $n = 0.035 \text{ s m}^{-1/3}$ for the main channel, and $n = 0.0446 \text{ s m}^{-1/3}$ for the floodplains.

The boundary conditions were obtained from Shamkhalchian and de Almeida (2021), and correspond to the time series of flow discharge and water surface elevation at the upstream and downstream sections of the river at the boundary of the computational domain. The initial

conditions $U(x, y, t = 0)$ were defined from the results of the FV model under steady-state conditions ($Q = 374 \text{ m}^3 \text{ s}^{-1}$) performed at 5 m resolution. This steady-state FV solution was obtained running the unsteady flow model with constant boundary conditions for long enough (i.e. $t = 10 \text{ h}$) for the steady state to be reached. PINNs were trained from datasets resolved at 50, 100 and 200 m, while the FV model was run using meshes generated from gridded data at resolutions of 10, 25 and 50 m. The corresponding temporal resolution for the trainset for the PINNs is 4 h. The batch size was set to one-third of the total number of collocation points.

For this test case, the architecture of the PICN consists of 2 convolutional layers (the first and second layers have 10 and 40 channels, respectively) and 1 fully connected hidden layer with 100 neurons. The architecture of the PIFCN consists of 3 fully connected hidden layers, each having 2000 neurons. Our tests showed that further increasing the network complexity would not significantly improve the model's prediction accuracy, and may substantially increase the training time and/or cause the program to exceed the memory capacity of the computer resources used.

Since an analytical solution is not available for this problem, the results of the FV simulation at fine resolution (5 m) were used as the benchmark. The accuracy of the solutions of the time-dependent variables is assessed at two cross-sections (located approximately at distances of 1/3 and 2/3 of the length of the river within the domain from the upstream boundary, and hereafter referred to as S1 and S2, respectively) at 1 h temporal resolution.

3.3.2. Results and analysis

Fig. 11 gives some examples of the predictions of water surface elevation η compared against the benchmarks at cross-sections S1.

Figs. 12 and 13 illustrate the time series of prediction errors (right vertical axes), along with the actual predicted values of the flow depth h and flow discharge Q (left vertical axes) at cross-sections S1 and S2 for each PICN, PIFCN, and FV models. Fig. 12 shows that the FV and PIFCN simulations consistently predict larger and lower depths than the benchmark solution, respectively, at both cross sections in the main channel, while PICN results display both positive and negative values of ϵ_h . Results from PICNs at S1 and S2 are markedly more accurate than those delivered by PIFCNs and the coarse-resolution FV models. For example, FV (50) and FV (25) produced results that deviate substantially (i.e. up to approximately 1.2 m and 2.5 m at S1 and S2, respectively) from the benchmark solution. On the other hand, FV (10) generally produced the most accurate depth predictions out of all models tested. The ability of the models to predict flow velocities (and therefore, the volumetric flow rate Q) is assessed by $\epsilon_Q = \bar{Q} - |Q|$, where $Q = \int_S h \bar{U} \cdot \mathbf{n} dl$ is the total discharge (where S denotes the length of the cross-section); l is the length along the cross-sections (i.e., S1 and S2, which span across the whole domain) and $\mathbf{n}$ is the unit vector normal to the cross-section. Fig. 13 shows the predicted errors ϵ_Q obtained by all models as a function of time. These results are markedly different from those previously presented for ϵ_h . Namely, all FV models display values of ϵ_Q that are substantially smaller than those predicted by PICN and PIFCN models. The maximum values of ϵ_Q for PICN and PIFCN are more than 50% and 70% of the benchmark (FV (5)) in S2, respectively. The possible reason behind these results might be that the water surface ($\eta = h + z$) presents much less spatial variation than discharges (i.e., hu and h^2v) in the domain. However, this hypothesis would need to be tested thoroughly in the future through a set of specifically designed case studies.

Fig. 14 assesses the overall accuracy of temporal prediction for h of each model against the corresponding computational time, using the metric $R'_h = \left(\sqrt{\sum (h_i - \bar{h}_i)^2 / N_p^t} \right)_{(x,y) \in S}$, where N_p^t is the number of collocation points in the testset. Table 3 also summarizes the results for each model. The best values of R'_h (i.e., across all epochs) obtained from all PICN models are within the range of $0.22 \text{ m} < R'_h < 0.30 \text{ m}$ (S1) and $0.26 \text{ m} < R'_h < 0.34 \text{ m}$ (S2), while FV (10) delivered $R'_h = 0.29 \text{ m}$ (S1) and 0.35 m (S2), and results from FV (25) and FV (50) were substantially less accurate. It is interesting to note that PINN models trained with coarse datasets (e.g., 200 m) do not necessarily deliver poorer accuracy compared to their fine-resolution counterparts; this contrasts with what is typically observed in simulations with traditional numerical methods such as FV. Fig. 14 also indicates that PICN models may offer improved depth predictions at lower cost than a FV model. For example, the accuracy of depth predictions by PICN (200) is better than the accuracy delivered by FV (10), while the computational cost is more than one order of magnitude lower. Overall, the PICN shows better h prediction performance than PIFCN and FV in terms of the speed-accuracy trade-off.

Fig. 15 shows examples of flood depth maps at $t = 32 \text{ h}$ obtained by the FV model at resolutions of 5 m and 25 m, along with those produced by PICN and PIFCN at 100 m resolved trainsets. As expected from the results presented in Fig. 12, FV (25) overestimates h during the peak time (which also translates into a larger flooded area), while the opposite is observed for PICN (100) and PIFCN (100). Further spatial analysis can be seen in Appendix B.

4. Concluding remarks

In this paper, two physics-informed neuronal networks (PINNs) were developed to predict the evolution of free-surface flows typically modeled by the shallow water equations (SWEs). The PINN formulation proposed in this paper eliminates the need for labeled data, which is typically required in supervised learning. This is achieved by defining a loss function that combines the SWEs, the boundary conditions (BCs) and initial conditions (ICs), allowing the trained PINN to serve as an

Table 3

Computation time and prediction accuracy of PICN, PIFCN and FV in the test 3. Note that the value of $\mathcal{T}_c$ indicates the training time when PINN achieve the optimal prediction accuracy.

Model	$\mathcal{T}_c$ (min)	S1		S2	
		R_h (m)	R_Q (m ³ s ⁻¹)	R_h (m)	R_Q (m ² s ⁻¹)
PICN(50)	59.4	0.26	136.1	0.34	278.9
PICN(100)	15.3	0.29	118.3	0.28	317.2
PICN(200)	5.3	0.22	100	0.26	380.7
PIFCN(50)	504.9	0.20	342.3	0.28	502.8
PIFCN(100)	127.9	0.20	267.7	0.21	547.2
PIFCN(200)	30.2	0.24	564.8	0.21	576.5
FV(10)	2576.0	0.29	83.0	0.35	78.4
FV(25)	83.3	0.64	60.6	0.83	95.9
FV(50)	8.6	0.93	18.0	1.40	82.4

alternative method for solving the SWEs. The two PINNs developed and tested here vary in their architecture and main features. The first is based on the fully connected neural network (PIFCN), and the second on the convolutional neural network (PICN) approach.

Three test cases were used to assess the accuracy and computational performance of each model, including two idealized flow problems for which analytical solutions are available, and one simulation of a real-world flood event over a relatively large-scale and complex topography domain. First, the trade-off between computational speed and accuracy in PINN relative to FV varies and depends on specific conditions. In the idealized problems, the PICN and PIFCN predictions achieved higher accuracy (lower R_h) than the Finite Volume (FV) solver employed for comparison. However, in these problems, PINNs generally took longer to reach the same prediction accuracy as the coarsely resolved FV model. For the real-world flooding problem, in general, PINNs were able to yield similarly accurate predictions of flow depths compared to finely resolved FV simulations. However, all FV models show much higher accuracy in their predictions of Q . For the spatial analysis of flow depths at the peak of the flood event, PINNs were able to produce flood maps with accuracy (relative to the benchmark finely resolved FV simulation) that is comparable to the results of FV models run at intermediate resolution (e.g., 25 m). Some of the PINN models (e.g., PICN at 100 and 200 m resolution) achieved the same level of accuracy as the 25 m resolution FV model at least one order of magnitude faster. Second, the prediction capability of PINNs may be less affected by changes in grid resolution than the FV solver, which may represent important advantages in real-world applications where finely resolved topographic data may not always be available. Third, results show that, in most circumstances, PICNs usually exhibit better performance in terms of speed-accuracy trade-off than PIFCNs. However, more comparative tests between PICN and PIFCN are necessary before reaching general conclusions in this regard.

While the results in this paper may not suggest that PINNs can replace other well-established numerical techniques, they indicate that PINNs (and in particular PICNs) should be considered as an emerging technique that has the potential to deliver accurate and efficient solutions, and which should be further developed and assessed. Our results show that the approach might be particularly useful under certain circumstances which are challenging to conventional techniques. For example, in simulations performed at coarse resolutions (a typical case in real-world problems), PINN models may achieve a higher prediction accuracy with a lower computational cost than a FV solver.

The use of Physics-informed Neural Network is a relatively new approach for the solution of partial differential equations. As is often the case with emerging technologies, the technique has a number of limitations, which are outlined here with the aim of guiding further research. First, prediction accuracy may be sensitive to model initialization, which deserves further research. Second, a single trained PINN can only solve the SWEs with a specific set of ICs and BCs. Therefore, each new problem (i.e. different sets of BCs and ICs) requires the network to

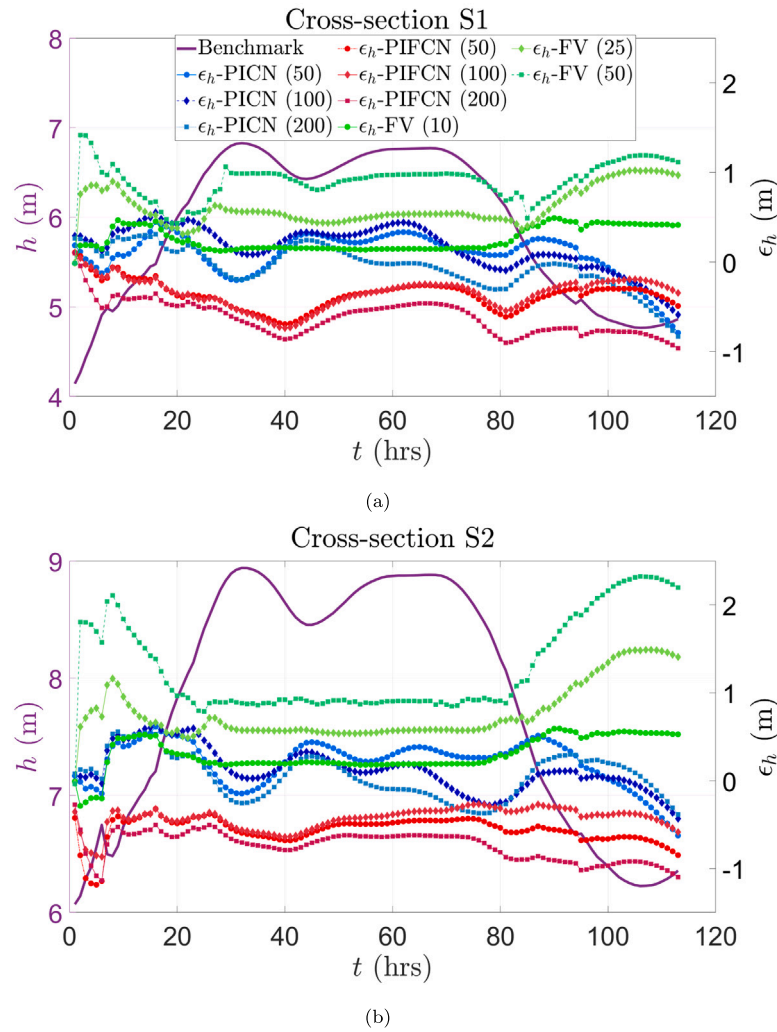


Fig. 12. Test 3: Predicted water depths error ϵ_h (plotted against right y-axis) at cross-sections S1 (a) and S2 (b) of the main channel in the Tiber river. Benchmark solution (from a finely resolved FV simulation) shown by the purple line against the left y-axis.

be retrained from scratch. Finally, defining an appropriate set of hyper-parameters (e.g. number of hidden layers and nodes, learning rate) for the solution of a given problem is a time-consuming process requiring several trial-and-error attempts before the final network architecture can be defined.

CRediT authorship contribution statement

Xin Qi: Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Gustavo A.M. de Almeida:** Writing – review & editing, Supervision, Software, Resources, Conceptualization. **Sergio Maldonado:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

Partial funding was received from the University of Southampton. XQ was partly funded by the University of Southampton through a Research Studentship. The authors acknowledge the use of the IRIDIS High Performance Computing Facility, and associated support services at the University of Southampton, in the completion of this work. The authors would also like to thank Dr Alireza Shamkhalchian for sharing data used for the simulation of the Tiber River flood event. The authors are grateful for the efforts made by three anonymous referees in reviewing this paper, which have helped improve the latter.

Appendix A. PINN design experiments

This section illustrates the heuristic approach followed to determine the best possible design of the PINNs. We focus on Test 1, described in Section 3.1. All the PINNs shown in this section are trained from the same dataset resolved at 50 m resolution. Figs. A.16 and A.17 show the accuracy ($\mathcal{R}_h$) of the PICNs and PIFCNs, respectively, as their architecture (number of layers and channels/neurons) is varied. In short, these figures show that it is difficult to conclude whether a single architecture can lead to significantly improved results, and we thus prioritize simplicity in our PINNs design. While this heuristic approach is, by definition, not guaranteed to find the optimal solution,

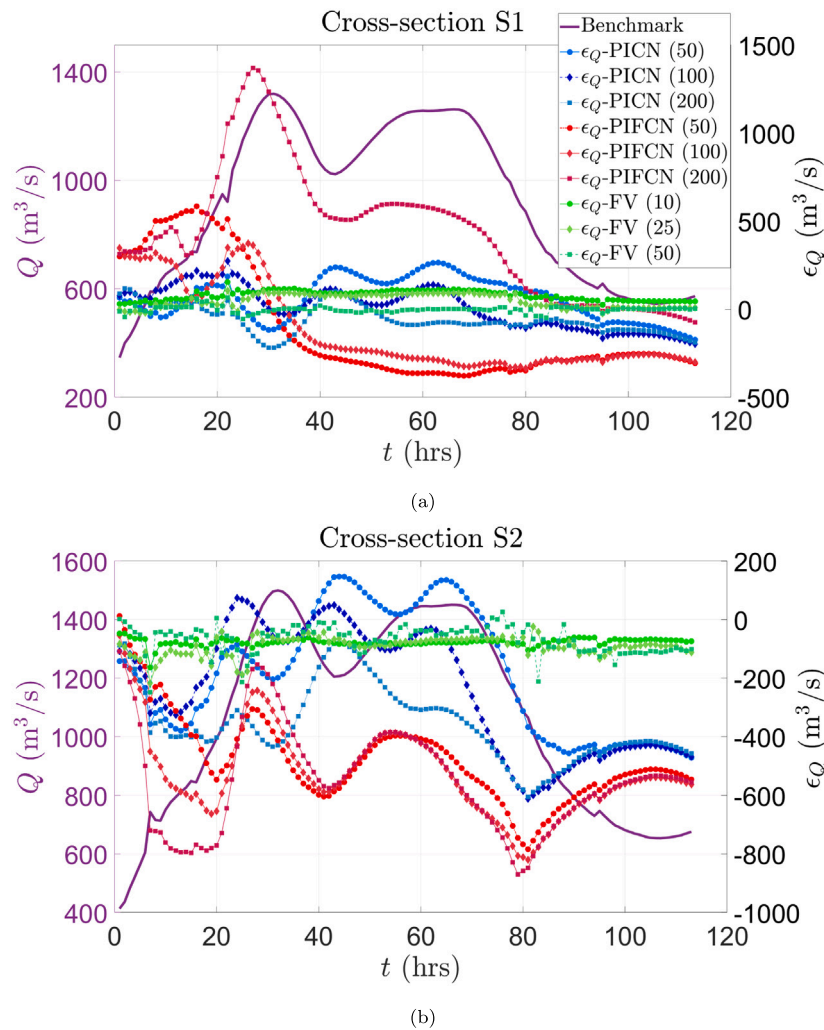


Fig. 13. Test 3: Predicted water discharge error ϵ_Q (plotted against right y-axis) at cross-sections S1 (a) and S2 (b) spanning across the whole domain in the Tiber river. Benchmark solution (from a finely resolved FV simulation) shown by the purple line against the left y-axis. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table A.4

Results of water depth prediction by using Relu, Sigmoid and Tanh activation functions for PICN and PIFCN models. The trainset is a 50 m resolved dataset from Test 1; the evaluation metric is $\mathcal{R}_h$.

Model	Relu	Sigmoid	Tanh
PICN	0.021 m	0.002 m	0.002 m
PIFCN	0.154 m	0.028 m	0.004 m

it represents the summary of many iterations. This holds for other tests and dataset resolutions considered in this study.

Similarly, we have tested three widely used activation functions: Relu, Sigmoid and Tanh (see Table A.4). The chosen architecture for testing the PICN and PIFCN models is CNN-5-20 and FCNN-3(1000), respectively. For PICN, Sigmoid and Tanh display the same accuracy, while the result of the Relu-based PICN has higher errors. The PIFCN with Tanh yields better accuracy than using the other two activation functions. As a result, Tanh was chosen as the activation function to be employed in all PINNs discussed in this paper.

Appendix B. Spatial variation analysis for peak floods in test 3

Tables B.5 and B.6 summarize the spatial prediction accuracy (i.e. $\mathcal{R}_h^s$, $\mathcal{R}_{hu}^s$, $\mathcal{R}_{hv}^s$) computed from a 50 m resolved set of points for each model at $t = 32$ and 68 h, as well as their overall $\mathcal{T}_c$ (i.e. training time for PINN and computation time for FV). Among all the models, FV (10) and FV (50) achieve the highest and lowest accuracy, respectively. All PINNs present lower $\mathcal{R}_h^s$ than FV (25) and FV (50). On the other hand, FV (25) is more accurate than all PINNs in terms of hu prediction. PIFCN show a relatively similar value of $\mathcal{R}_{hu}^s$ to FV (50) at both time points. Moreover, the prediction accuracy of the PICNs and PIFCNs is less affected by the resolution of the input dataset than in the FV model. This last point may potentially be a main advantage of PINNs relative to conventional numerical methods in general, whose performance (numerical stability and accuracy) tends to be highly dependent on the mesh resolution.

Appendix C. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.jhydrol.2024.131263>.

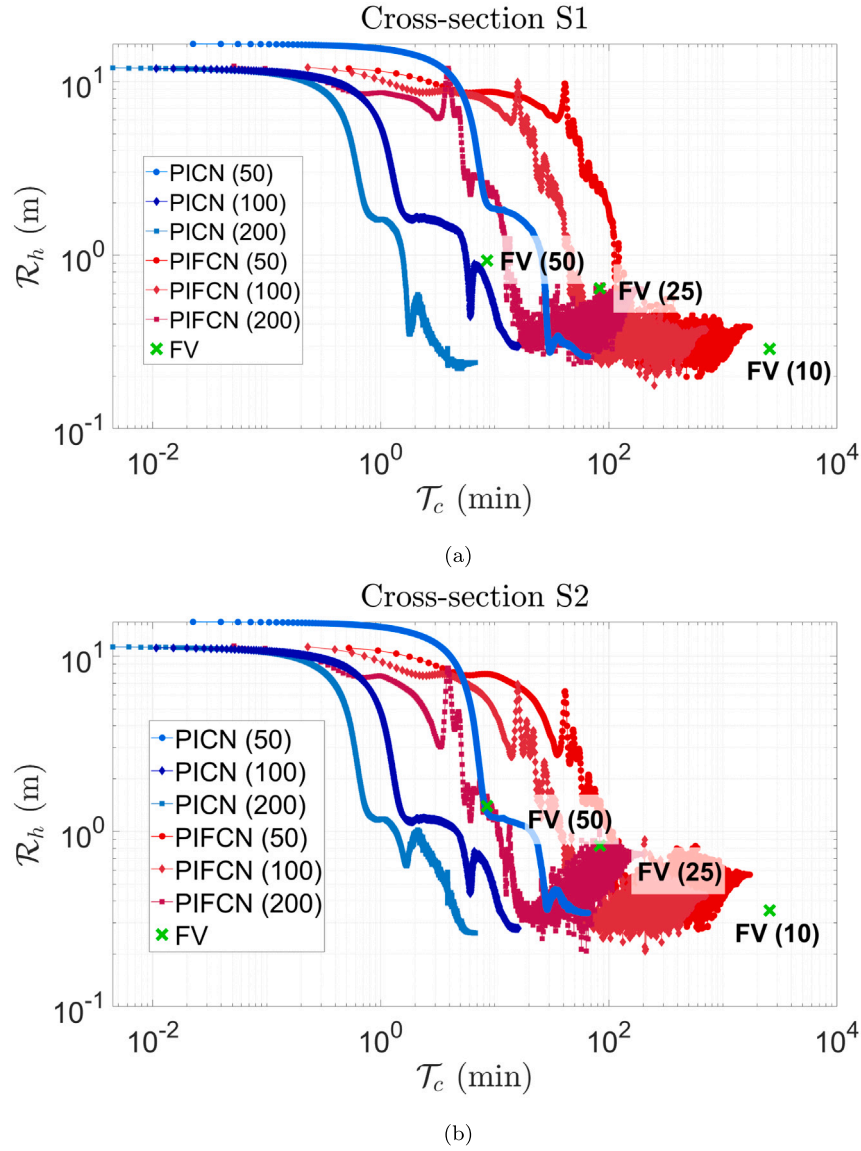


Fig. 14. Test 3: R_h^s as a function of T_c (training time for PICN and PIFCN and run time for FV) at cross-sections S1 (a) and S2 (b) of the Tiber river; note that the right-most point of each cloud corresponds to the highest accuracy that any given PINN can achieve. The number in brackets represents the resolution (in meters) of the training data set (for PICN and PIFCN) or mesh (for the FV model). The benchmark results are those from the FV (5) simulation.

Table B.5

Computation time and spatial prediction accuracy relative to benchmark simulation for the comparison at $t = 32$ h.

Model	T_c (min)	R_h^s (m)	R_{hu}^s ($\text{m}^2 \text{s}^{-1}$)	R_{hv}^s ($\text{m}^2 \text{s}^{-1}$)
PICN (50)	59.4	0.52	1.68	1.16
PICN (100)	15.3	0.40	1.72	1.18
PICN (200)	5.3	0.48	1.69	1.12
PIFCN (50)	504.9	0.59	2.21	1.32
PIFCN (100)	127.9	0.59	2.16	1.28
PIFCN (200)	30.2	0.63	2.27	1.21
FV (10)	2576.0	0.19	0.89	0.56
FV (25)	83.3	0.64	1.20	1.25
FV (50)	8.6	1.24	2.18	1.95

Table B.6

Spatial prediction accuracy relative to benchmark simulation for the comparison at $t = 68$ h.

Model	R_h^s (m)	R_{hu}^s ($\text{m}^2 \text{s}^{-1}$)	R_{hv}^s ($\text{m}^2 \text{s}^{-1}$)
PICN (50)	0.41	1.87	1.21
PICN (100)	0.37	1.92	1.20
PICN (200)	0.39	1.88	1.17
PIFCN (50)	0.52	2.21	1.28
PIFCN (100)	0.50	2.21	1.18
PIFCN (200)	0.47	2.16	1.15
FV (10)	0.19	0.86	0.56
FV (25)	0.64	1.36	1.21
FV (50)	1.24	2.32	1.87

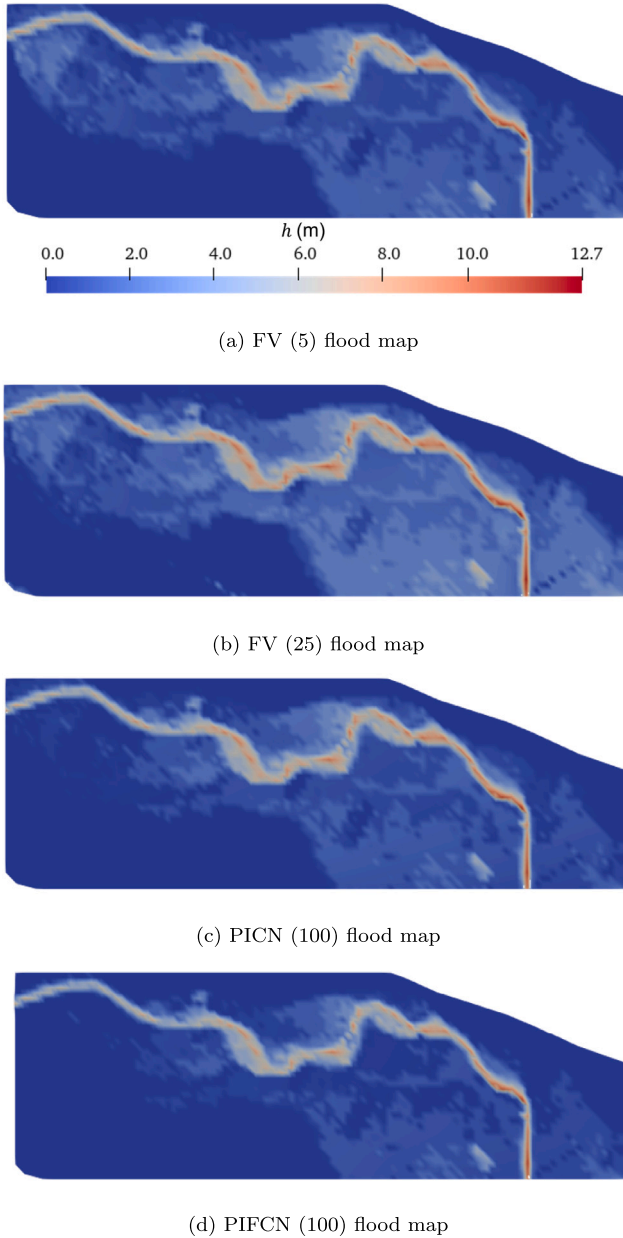


Fig. 15. Examples of flood maps at time $t = 32$ h produced by the FV model and PINNs. FV (5) represents the benchmark results. Note that all the maps share the same color bar (below FV (5) map) and are plotted at 50 m spatial resolution. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

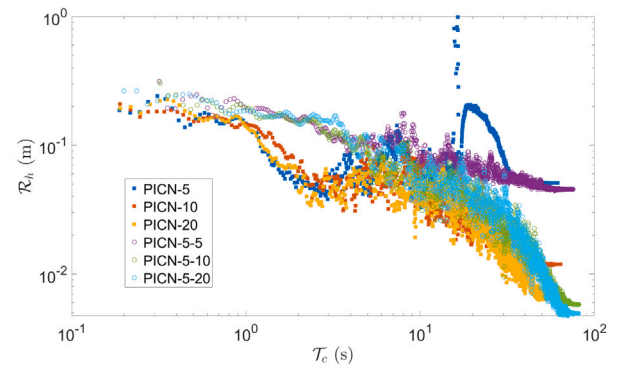


Fig. A.16. Comparison of PICNs with different architectures; the last hidden layer of all PICNs is one typical fully connected layer with 50 neurons. In the legend bar, the following format is adopted: PICN-X-Y, where the PICN has X channels in the first convolutional layer and Y channels in the second convolutional layer (thus, PICN-X denotes a network with one convolutional layer only).

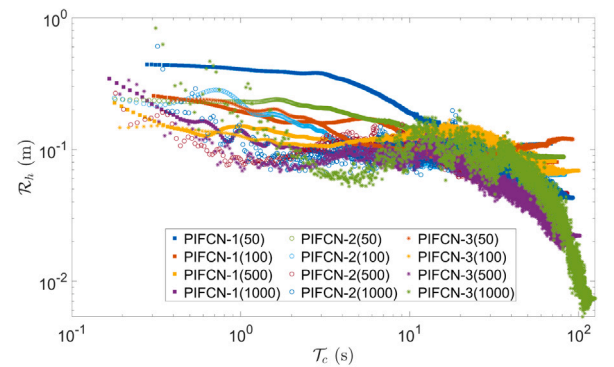


Fig. A.17. Comparison of PIFCNs with different architectures. In the legend bar, the following format is adopted: PIFCN-X(Y), where X denotes the number of hidden layers and Y is the number of neurons per layer.

References

- Alcrudo, F., Garcia-Navarro, P., 1993. A high-resolution Godunov-type scheme in finite volumes for the 2D shallow-water equations. *Internat. J. Numer. Methods Fluids* 16 (6), 489–505.
- Bale, D.S., Leveque, R.J., Mitran, S., Rossmannith, J.A., 2003. A wave propagation method for conservation laws and balance laws with spatially varying flux functions. *SIAM J. Sci. Comput.* 24 (3), 955–978.
- Baydin, A.G., Pearlmutter, B.A., Radul, A.A., Siskind, J.M., 2018. Automatic differentiation in machine learning: A survey. *J. March. Learn. Res.* 18, 1–43.
- Bermúdez, M., Cea, L., Puertas, J., 2019. A rapid flood inundation model for hazard mapping based on least squares support vector machine regression. *J. Flood Risk Manag.* 12 (August 2018), 1–14. <http://dx.doi.org/10.1111/jfr3.12522>, Times cited: 6 Flooding papers.
- Bernard, P.-E., Remacle, J.-F., Comblen, R., Legat, V., Hillewaert, K., 2009. High-order discontinuous Galerkin schemes on general 2D manifolds applied to the shallow water equations. *J. Comput. Phys.* 228 (17), 6514–6535.
- Bihlo, A., Popovych, R.O., 2022. Physics-informed neural networks for the shallow-water equations on the sphere. *J. Comput. Phys.* 456, 111024. <http://dx.doi.org/10.1016/j.jcp.2022.111024>.
- Blumer, A., Ehrenfeucht, A., Haussler, D., Warmuth, M.K., 1987. Occam's Razor. *Inform. Process. Lett.* 24 (6), 377–380. [http://dx.doi.org/10.1016/0020-0190\(87\)90114-1](http://dx.doi.org/10.1016/0020-0190(87)90114-1).
- Botta, N., Klein, R., Langenberg, S., Lützenkirchen, S., 2004. Well balanced finite volume methods for nearly hydrostatic flows. *J. Comput. Phys.* 196 (2), 539–565.
- Cai, S., Wang, Z., Wang, S., Perdikaris, P., Karniadakis, G.E., 2021. Physics-informed neural networks for heat transfer problems. *J. Heat Transfer* 143 (6).
- Casulli, V., 1990. Semi-implicit finite difference methods for the two-dimensional shallow water equations. *J. Comput. Phys.* 86 (1), 56–74.
- Dawson, C., Westerink, J.J., Feyen, J.C., Pothina, D., 2006. Continuous, discontinuous and coupled discontinuous-continuous Galerkin finite element methods for the shallow water equations. *Internat. J. Numer. Methods Fluids* 52 (1), 63–88.
- de Almeida, G.A., Bates, P., 2013. Applicability of the local inertial approximation of the shallow water equations to flood modeling. *Water Resour. Res.* 49 (8), 4833–4844.

- de Almeida, G.A., Bates, P., Ozdemir, H., 2016. Modelling urban floods at submetre resolution: Challenges or opportunities for flood risk management? *J. Flood Risk Manag.* 11, S855–S865.
- Debbarma, S., Dey, S., Bandyopadhyay, A., Bhadra, A., 2024. Simulation of flood inundation extent by integration of HEC-HMS, GA-based rating curve and cost distance analysis. *Water Resour. Manag.* 1–21.
- Delestre, O., Lucas, C., Ksinant, P.-A., Darboux, F., Laguerre, C., Vo, T.-N.-T., James, F., Cordier, S., 2013. SWASHES: A compilation of shallow water analytic solutions for hydraulic and environmental studies. *Internat. J. Numer. Methods Fluids* 72 (3), 269–300. <http://dx.doi.org/10.1002/fld.3741>.
- Donnelly, J., Daneshkhan, A., Abolfathi, S., 2024a. Forecasting global climate drivers using Gaussian processes and convolutional autoencoders. *Eng. Appl. Artif. Intell.* 128, 107536.
- Donnelly, J., Daneshkhan, A., Abolfathi, S., 2024b. Physics-informed neural networks as surrogate models of hydrodynamic simulators. *Sci. Total Environ.* 912, 168814.
- Feng, D., Tan, Z., He, Q., 2023. Physics-informed neural networks of the Saint-Venant equations for downscaling a large-scale river model. *Water Resour. Res.* 59 (2), e2022WR033168.
- Ferrari, A., Vacondio, R., 2022. An augmented HLEM ADER numerical model parallel on GPU for the porous shallow water equations. *Comput. & Fluids* 238, 105360.
- Fraehr, N., Wang, Q.-J., Wu, W., Nathan, R., 2024. Assessment of surrogate models for flood inundation: The physics-guided LSG model vs. state-of-the-art machine learning models. *Water Res.* 252, 121202.
- Gao, H., Sun, L., Wang, J.-X., 2021. PhyGeoNet: Physics-informed geometry-adaptive convolutional neural networks for solving parameterized steady-state PDEs on irregular domain. *J. Comput. Phys.* 428, 110079.
- Guo, L., Ye, S., Han, J., Zheng, H., Gao, H., Chen, D.Z., Wang, J.-X., Wang, C., 2020. SSR-VFD: Spatial super-resolution for vector field data analysis and visualization. In: 2020 IEEE Pacific Visualization Symposium. *PacificVis*, pp. 71–80. <http://dx.doi.org/10.1109/PacificVis48177.2020.8737>.
- Gurbuz, F., Mudiredy, A., Mantilla, R., Xiao, S., 2024. Using a physics-based hydrological model and storm transposition to investigate machine-learning algorithms for streamflow prediction. *J. Hydrol.* 628, 130504.
- Habib, M., O'Sullivan, J., Abolfathi, S., Salauddin, M., 2023. Enhanced wave overtopping simulation at vertical breakwaters using machine learning algorithms. *PLoS One* 18 (8), e0289318.
- Haghighat, E., Raissi, M., Moure, A., Gomez, H., Juanes, R., 2020. A deep learning framework for solution and discovery in solid mechanics. *arXiv preprint arXiv:2003.02751*.
- Hanert, E., Le Roux, D.Y., Legat, V., Deleersnijder, E., 2005. An efficient Eulerian finite element method for the shallow water equations. *Ocean Model.* 10 (1–2), 115–136.
- Haykin, S., 2009. *Neural networks and learning machines*, Third Edit Prentice-Hall, Upper Saddle River.
- Hornik, K., Stinchcombe, M., White, H., 1989. Multilayer feedforward networks are universal approximators. *Neural Netw.* 2 (5), 359–366.
- Hunter, N.M., Horritt, M.S., Bates, P.D., Wilson, M.D., Werner, M.G., 2005. An adaptive time step solution for raster-based storage cell modelling of floodplain inundation. *Adv. Water Resour.* 28 (9), 975–991.
- Ioffe, S., Szegedy, C., 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *CoRR abs/1502.03167*, [arXiv:1502.03167](https://arxiv.org/abs/1502.03167).
- Jagtap, N.V., Mudunuru, M., Nakshatrala, K., 2023. CoolPINNs: A physics-informed neural network modeling of active cooling in vascular systems. *Appl. Math. Model.* 122, 265–287.
- Jin, X., Cai, S., Li, H., Karniadakis, G.E., 2021. Nsfnets (Navier-Stokes flow nets): Physics-informed neural networks for the incompressible Navier-Stokes equations. *J. Comput. Phys.* 426, 109951.
- Juez, C., Murillo, J., García-Navarro, P., 2014. A 2D weakly-coupled and efficient numerical model for transient shallow flow and movable bed. *Adv. Water Resour.* 71, 93–109.
- Kabir, S., Patidar, S., Xia, X., Liang, Q., Neal, J., Pender, G., 2020. A deep convolutional neural network model for rapid prediction of fluvial flood inundation. *J. Hydrol.* 590, 125481.
- Khan, S., Yairi, T., 2018. A review on the application of deep learning in system health management. *Mech. Syst. Signal Process.* 107, 241–265.
- Kharazmi, E., Zhang, Z., Karniadakis, G.E., 2021. Hp-VPINNs: Variational physics-informed neural networks with domain decomposition. *Comput. Methods Appl. Mech. Engrg.* 374, 113547.
- Kingma, D.P., Ba, J., 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Krishnapriyan, A., Gholami, A., Zhe, S., Kirby, R., Mahoney, M.W., 2021. Characterizing possible failure modes in physics-informed neural networks. *Adv. Neural Inf. Process. Syst.* 34.
- Kurganov, A., Levy, D., 2002. Central-upwind schemes for the Saint-Venant system. *ESAIM Math. Model. Numer. Anal.* 36 (3), 397–425.
- Leandro, J., Chen, A., Schumann, A., 2014. A 2D parallel diffusive wave model for floodplain inundation with variable time step (P-DWave). *J. Hydrol.* 517, 250–259.
- Leskens, J., Brugnach, M., Hoekstra, A.Y., Schuurmans, W., 2014. Why are decisions in flood disaster management so poorly supported by information from flood models? *Environ. Modell. Softw.* 53, 53–61.
- LeVeque, R.J., George, D.L., Berger, M.J., 2011. Tsunami modelling with adaptively refined finite volume methods. *Acta Numer.* 20, 211–289.
- Li, J., Cao, Z., Borthwick, A.G., 2022. Quantifying multiple uncertainties in modelling shallow water-sediment flows: A stochastic Galerkin framework with Haar wavelet expansion and an operator-splitting approach. *Appl. Math. Model.* 106, 259–275. <http://dx.doi.org/10.1016/j.apm.2022.01.032>, URL <https://www.sciencedirect.com/science/article/pii/S0307904X22000579>.
- Li, C., Han, Z., Li, Y., Li, M., Wang, W., Dou, J., Xu, L., Chen, G., 2023. Physical information-fused deep learning model ensemble with a subregion-specific sampling method for predicting flood dynamics. *J. Hydrol.* 620, 129465. <http://dx.doi.org/10.1016/j.jhydrol.2023.129465>, URL <https://www.sciencedirect.com/science/article/pii/S0022169423004079>.
- Li, R., Lee, E., Luo, T., 2021. Physics-informed neural networks for solving multiscale mode-resolved phonon Boltzmann transport equation. *Mater. Today Phys.* 19, 100429.
- Liang, Q., 2011. A structured but non-uniform Cartesian grid-based model for the shallow water equations. *Internat. J. Numer. Methods Fluids* 66 (5), 537–554.
- Liang, Q., Marche, F., 2009. Numerical resolution of well-balanced shallow water equations with complex source terms. *Adv. Water Resour.* 32 (6), 873–884.
- Liu, Y., Pender, G., 2015. A flood inundation modelling using v-support vector machine regression model. *Eng. Appl. Artif. Intell.* 46, 223–231. <http://dx.doi.org/10.1016/j.engappai.2015.09.014>, Times cited: 4 Flooding papers.
- Lynch, D.R., Gray, W.G., 1979. A wave equation model for finite element tidal computations. *Comput. & Fluids* 7 (3), 207–228.
- MacDonald, I., 1996. *Analysis and Computation of Steady Open Channel Flow* (Ph.D. thesis). Citeseer.
- Mahesh, R.B., Leandro, J., Lin, Q., 2022. Physics informed neural network for spatial-temporal flood forecasting. In: *Climate Change and Water Security*. Springer, pp. 77–91.
- Mao, Z., Jagtap, A.D., Karniadakis, G.E., 2020. Physics-informed neural networks for high-speed flows. *Comput. Methods Appl. Mech. Engrg.* 360, 112789.
- Marras, S., Kelly, J.F., Moragues, M., Müller, A., Kopera, M.A., Vázquez, M., Giraldo, F.X., Houzeaux, G., Jorba, O., 2016. A review of element-based Galerkin methods for numerical weather prediction: Finite elements, spectral elements, and discontinuous Galerkin. *Arch. Comput. Methods Eng.* 23 (4), 673–722.
- Molls, T., Chaudhry, M.H., 1995. Depth-averaged open-channel flow model. *J. Hydraul. Eng.* 121 (6), 453–465.
- Monnier, J., Couderc, F., Dartus, D., Larnier, K., Madec, R., Vila, J.-P., 2016. Inverse algorithms for 2D shallow water equations in presence of wet dry fronts: Application to flood plain dynamics. *Adv. Water Resour.* 97, 11–24.
- Morales-Hernández, M., Petaccia, G., Brufau, P., García-Navarro, P., 2016. Conservative 1D–2D coupled numerical strategies applied to river flooding: The Tiber (Rome). *Appl. Math. Model.* 40 (3), 2087–2105.
- Pang, G., Lu, L., Karniadakis, G.E., 2019. fPINNs: Fractional physics-informed neural networks. *SIAM J. Sci. Comput.* 41 (4), A2603–A2626.
- Paszke, A., Gross, S., Chintala, S., Chanan, G., Yang, E., DeVito, Z., Lin, Z., Desmaison, A., Antiga, L., Lerer, A., 2017. Automatic differentiation in pytorch.
- Raissi, M., Perdikaris, P., Karniadakis, G.E., 2019. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* 378, 686–707.
- Rastgo, R., Kiani, K., Escalera, S., Sabokrou, M., 2021. Sign language production: A review. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. pp. 3451–3461.
- Sanders, B.F., Schubert, J.E., 2019. PRIMO: Parallel raster inundation model. *Adv. Water Resour.* 126, 79–95.
- Shamkhalchian, A., de Almeida, G.A., 2021. Upscaling the shallow water equations for fast flood modelling. *J. Hydraul. Res.* 59 (5), 739–756.
- Smith, L.N., Topin, N., 2019. Super-convergence: Very fast training of neural networks using large learning rates. In: *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, vol. 11006, International Society for Optics and Photonics, 1100612.
- Sun, L., Gao, H., Pan, S., Wang, J.-X., 2020. Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data. *Comput. Methods Appl. Mech. Engrg.* 361, 112732.
- Toro, E.F., García-Navarro, P., 2007. Godunov-type methods for free-surface shallow flows: A review. *J. Hydraul. Res.* 45 (6), 736–751.
- Vlassis, N.N., Sun, W., 2021. Sobolev training of thermodynamic-informed neural networks for interpretable elasto-plasticity models with level set hardening. *Comput. Methods Appl. Mech. Engrg.* 377, 113695.
- Voulodimos, A., Doulamis, N., Doulamis, A., Protopapadakis, E., 2018. Deep learning for computer vision: A brief review. *Comput. Intell. Neurosci.* 2018.
- Wilson, M., Bates, P., Alsdorf, D., Forsberg, B., Horritt, M., Melack, J., Frappart, F., Famiglietti, J., 2007. Modeling large-scale inundation of Amazonian seasonally flooded wetlands. *Geophys. Res. Lett.* 34 (15).
- Wu, C., Zhu, M., Tan, Q., Kartha, Y., Lu, L., 2023. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. *Comput. Methods Appl. Mech. Engrg.* 403, 115671.
- Yeganeh-Bakhtiary, A., Eyvazoghli, H., Shabakhty, N., Abolfathi, S., 2023. Machine learning prediction of wave characteristics: Comparison between semi-empirical approaches and DT model. *Ocean Eng.* 286, 115583.
- Yeganeh-Bakhtiary, A., Eyvazoghli, H., Shabakhty, N., Kamranzad, B., Abolfathi, S., 2022. Machine learning as a downscaling approach for prediction of wind characteristics under future climate change scenarios. *Complexity* 2022.

- Yoon, T.H., Kang, S.-K., 2004. Finite volume model for two-dimensional shallow water flows on unstructured grids. *J. Hydraul. Eng.* 130 (7), 678–688.
- Zhang, R., Liu, Y., Sun, H., 2020. Physics-guided convolutional neural network (PhyCNN) for data-driven seismic response modeling. *Eng. Struct.* 215, 110704.
- Zhu, X.X., Tuia, D., Mou, L., Xia, G.-S., Zhang, L., Xu, F., Fraundorfer, F., 2017. Deep learning in remote sensing: A comprehensive review and list of resources. *IEEE Geosci. Remote Sens. Mag.* 5 (4), 8–36.
- Zhu, Y., Zabaras, N., Koutsourelakis, P.-S., Perdikaris, P., 2019. Physics-constrained deep learning for high-dimensional surrogate modeling and uncertainty quantification without labeled data. *J. Comput. Phys.* 394, 56–81.